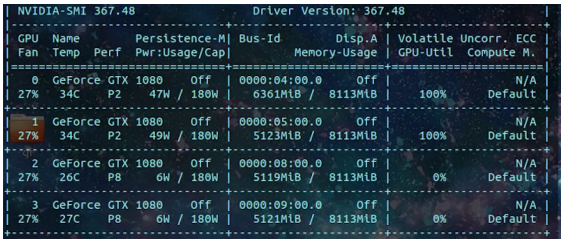
How to Train a Model Using Multiple GPUs in Pytorch

Training models with GPUs helps significantly shorten the training time compared to using CPUs

Lin Wang · Follow
12 min read · Sep 30, 2024



If you are someone who studies and works in AI, you are probably already familiar with training/testing models using GPUs. Training models with GPUs helps significantly shorten the training time compared to using CPUs. When models or datasets become larger, using only one GPU is not enough. For example, current large language models require many powerful GPUs and need to be trained for many days, sometimes even months. Therefore, we need to find ways to train models on multiple GPUs at once, with the goal of speeding up the training process. PyTorch supports two methods to distribute models and data across multiple GPUs: `nn.DataParallel` and `nn.DistributedDataParallel`. In this article, we will explore how to efficiently train a model on multiple GPUs using PyTorch. Additionally, to best understand the content of this article, it's important to have knowledge of topics such as multiprocessing, neural networks, etc.

Introduction to `nn.DataParallel` and `nn.DistributedDataParallel`

The question arises: Should we use `nn.DataParallel` or `nn.DistributedDataParallel`? To answer this, we need to compare the differences between `DataParallel` and `DistributedDataParallel`. 🤩 We can note some key points as follows:

DataParallel:

- `DataParallel` operates in a single-process, multi-thread environment on a single machine.
- The main idea of `DataParallel` is to replicate the model across multiple GPUs, and each GPU handles a portion of the input data during each training iteration.
- However, using multi-threading on one machine may encounter GIL (Global Interpreter Lock) contention, which can reduce training performance.
- The model is duplicated across GPUs, which requires multiple copies of the model in memory, leading to memory consumption and the time cost of copying data between GPUs.

DistributedDataParallel:

- `DistributedDataParallel` operates in a multi-process environment and can run on both single-machine and multi-machine setups.
- The main idea of `DistributedDataParallel` is to use inter-process communication to distribute the training process across multiple GPUs and machines.
- This method minimizes the performance cost due to GIL contention and doesn't require multiple copies of the model to be stored. `DistributedDataParallel` divides the data and model across multiple GPUs and machines through efficient network communication, helping to accelerate training.

Using DataParallel:

In this section, we will design a mock training program to explore how to use `DataParallel`.

First, we will declare the necessary libraries and parameters:

```
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader

# Parameters and DataLoaders
input_size = 5
output_size = 2

batch_size = 30
data_size = 100
```

Device:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

To keep things simple, we will create a random dataset as follows:

```
class RandomDataset(Dataset):

    def __init__(self, size, length):
        self.len = length
        self.data = torch.randn(length, size)

    def __getitem__(self, index):
        return self.data[index]

    def __len__(self):
        return self.len

rand_loader = DataLoader(dataset=RandomDataset(input_size, data_size),
                         batch_size=batch_size, shuffle=True)
```

The model we use here is a neural network consisting of a single linear layer. You can also use other models for your experiments. To observe how


```
class Model(nn.Module):
    def __init__(self, input_size, output_size):
        super(Model, self).__init__()
        self.fc = nn.Linear(input_size, output_size)

    def forward(self, input):
        output = self.fc(input)
        print("\tIn Model: input size", input.size(),
              "output size", output.size())

        return output
```

Now, here comes the “main event” 😄 First, we will create a model instance and check how many GPUs we have. If the number of GPUs is greater than 1, we can wrap the model using `nn.DataParallel`. After that, we can declare the model to use the GPU with `model.to(device)`.

```
model = Model(input_size, output_size)
if torch.cuda.device_count() > 1:
  print("Let's use", torch.cuda.device_count(), "GPUs!")
  model = nn.DataParallel(model)

model.to(device)
```

Output:

```
Let's use 2 GPUs!

DataParallel(
  (module): Model(
    (fc): Linear(in_features=5, out_features=2, bias=True)
  )
)
```

We will check the size of the input tensor and output tensor as follows:

```
for data in rand_loader:
    input = data.to(device)
    output = model(input)
    print("Outside: input size", input.size(),
          "output_size", output.size())
```

Since we currently have 2 GPUs, the result will be as follows:

```
Let's use 2 GPUs!
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
Outside: input size torch.Size([30, 5]) output_size torch.Size([30, 2])
```

```
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
Outside: input size torch.Size([30, 5]) output_size torch.Size([30, 2])
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
    In Model: input size torch.Size([15, 5]) output size torch.Size([15, 2])
Outside: input size torch.Size([30, 5]) output_size torch.Size([30, 2])
    In Model: input size torch.Size([5, 5]) output size torch.Size([5, 2])
    In Model: input size torch.Size([5, 5]) output size torch.Size([5, 2])
Outside: input size torch.Size([10, 5]) output_size torch.Size([10, 2])
```

The size of the input tensor for each batch is [30, 5], but when passed into the model, it is evenly split into [15, 5] (because there are 2 GPUs) 😄. This means the data is divided into 2 parts corresponding to the 2 GPUs (batch_size / gpus). You can see that coding with `DataParallel` is very simple; we only need to add a single line of code, and it works 😄.

Model Parallel on a Single Machine

In the previous section, we used `DataParallel` to train a model on multiple GPUs. `DataParallel` replicates the model across all GPUs, and each GPU uses a different part of the input data. Although this method can somewhat speed up the training, it won't work in cases where the model is too large to run on a single GPU 😄. In such cases, instead of replicating the entire model across all GPUs, we "split" the model into smaller parts. For example, let's assume we have a model `m` with 10 layers. If we use `DataParallel` on 2 GPUs, each GPU will contain all 10 layers of the model `m`. However, if we use **model parallelism** on 2 GPUs, each GPU will only contain 5 layers of the model `m` 😄.

Generally speaking, the main idea of **model parallelism** is to divide the model into smaller parts (sub-networks) and place them on different computational devices, such as splitting the model into smaller sub-models on each separate GPU. Then, we use the "forward" function to move the intermediate output data between the devices. Since only a portion of the model runs on each specific computational device, a set of devices can work together to handle a larger model. This maximizes the computational power of the devices and solves the problem of resource limitations on individual devices.

We will start with a simple model consisting of 2 linear layers. To run the model on 2 GPUs, we will place each linear layer on a different GPU, then move the input and intermediate output to match the corresponding GPUs.

```
import torch
import torch.nn as nn
import torch.optim as optim

class RandomModel(nn.Module):
    def __init__(self):
        super(RandomModel, self).__init__()
        self.net1 = torch.nn.Linear(10, 5).to('cuda:0')
        self.relu = torch.nn.LeakyReLU()
        self.net2 = torch.nn.Linear(5, 5).to('cuda:1')

    def forward(self, x):
        x = self.relu(self.net1(x.to('cuda:0')))
        return self.net2(x.to('cuda:1'))
```

Here, `x` has been moved to `cuda:0` in `net1` and to `cuda:1` in `net2`, respectively. The implementation doesn't differ much from using a single GPU, apart from adding `to(device)` 😄. The `backward()` function and `torch.optim` will automatically compute gradients as they do on a single GPU. One important note is that we need to ensure the label and output are on the same device to calculate the loss for the model.

```
model = RandomModel()
loss_fn = nn.MSELoss()

optimizer = optim.SGD(model.parameters(), lr=0.001)
optimizer.zero_grad()

outputs = model(torch.randn(20, 10))
labels = torch.randn(20, 5).to('cuda:1')
loss_fn(outputs, labels).backward()
optimizer.step()
```

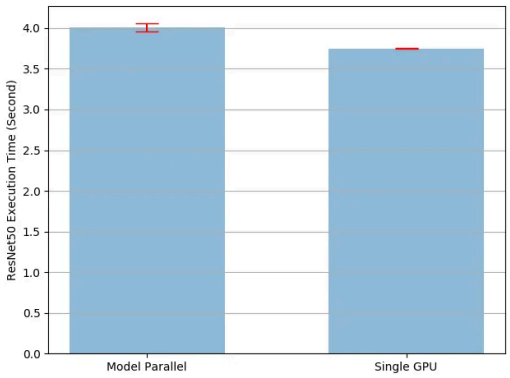
We can also apply model parallelism to an existing model by inheriting and overriding the layers and the `forward` function of that model. Here's an example of the code below:

```
from torchvision.models.resnet import ResNet, Bottleneck

num_classes = 2

class ModelParallelResNet50(ResNet):
    def __init__(self, *args, **kwargs):
        super(ModelParallelResNet50, self).__init__(
            Bottleneck, [3, 4, 6, 3], num_classes=num_classes, *args, **kwargs)

        self.seq1 = nn.Sequential(
            self.conv1,
```

```
            self.bn1,
            self.relu,
            self.maxpool,

            self.layer1,
            self.layer2
        ).to('cuda:0')

        self.seq2 = nn.Sequential(
            self.layer3,
            self.layer4,
            self.avgpool,
        ).to('cuda:1')

        self.fc.to('cuda:1')

    def forward(self, x):
        x = self.seq2(self.seq1(x).to('cuda:1'))
        return self.fc(x.view(x.size(0), -1))
```

In the code above, we import the ResNet model from `torchvision.models.resnet`. To run this model in parallel and distribute it across 2 GPUs, we will create a new class named `ModelParallelResNet50` that inherits from ResNet. Then, we will modify the properties of `seq1`, `seq2`, and `fc` to be moved to different devices. Next, we will override the `forward` function to combine the two components of the network by transferring the intermediate output to the corresponding devices.

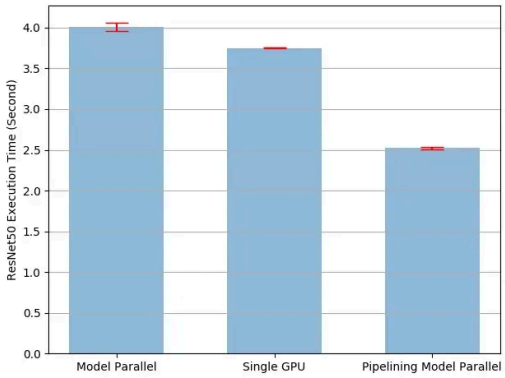
The method described may result in slower training compared to running solely on a single GPU. This is because only one of the two GPUs is active at any given time, and it takes time to continuously copy the intermediate outputs from `cuda:0` to `cuda:1`. Experimental results from the `model_parallel_tutorial` show that using model parallelism is 7% slower than running on a single GPU. This also accounts for the cost of copying tensors between GPUs.



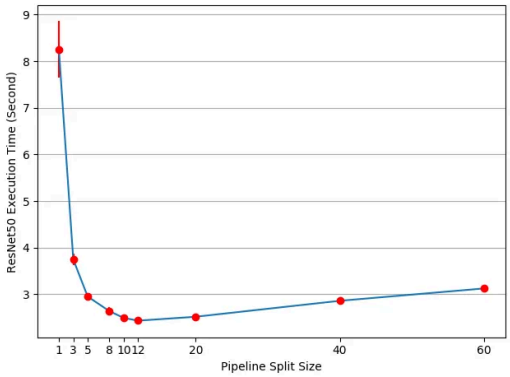
One way to solve the problem of one GPU doing all the work while the other sits idle 😊 is to split the data batch so that when one part of the data reaches the second sub-network, the remaining part is fed into the first sub-network. This way, we can run simultaneously on both GPUs.

Suppose we initially run with a batch size of 100; we will split the batch into parts containing 20 images each (batch of batches 😊). The implementation will be as follows:

```
class PipelineParallelResNet50(ModelParallelResNet50):
    def __init__(self, split_size=20, *args, **kwargs):
        super(PipelineParallelResNet50, self).__init__(*args, **kwargs)
        self.split_size = split_size

    def forward(self, x):
        splits = iter(x.split(self.split_size, dim=0))
        s_next = next(splits)
        s_prev = self.seq1(s_next).to('cuda:1')
        ret = []

        for s_next in splits:
            # A. ``s_prev`` runs on ``cuda:1``
            s_prev = self.seq2(s_prev)
            ret.append(self.fc(s_prev.view(s_prev.size(0), -1)))

            # B. ``s_next`` runs on ``cuda:0``, which can run concurrently with
            s_prev = self.seq1(s_next).to('cuda:1')

        s_prev = self.seq2(s_prev)
        ret.append(self.fc(s_prev.view(s_prev.size(0), -1)))

        return torch.cat(ret)
```



The speed has increased significantly compared to before. The question arises: what is the optimal value for `split_size`? Visually, using a small `split_size` leads to launching many small CUDA kernels, while using a large `split_size` results in relatively long "idle" times during the first and last splits. We have experimental results with different values of `split_size` as follows:



The execution speed is highest when `split_size` has a value of 12. Since this value depends on the model and data, there is no universally optimal solution for all cases.

Using DistributedDataParallel

Before we start using `DistributedDataParallel`, we need to understand some terms:

- **Node**: In a distributed architecture, a “node” is a single computing system capable of processing and containing multiple GPUs. This can be a single computer or a cluster of computers capable of parallel computing with multiple GPUs.
- **Global rank**: This is a unique identifier for each “node” in the distributed architecture. It helps distinguish each “node” uniquely within the system.
- **Local rank**: This is also a unique identifier, but it identifies the processes within each individual “node”.
- **World**: The “world” is the set of all entities in the distributed architecture, including both “nodes” and processes within each “node”. It serves as a global workspace for the entire distributed system.
- **world_size**: This is a number that determines the size of the “world,” i.e., the total number of processes in the system. It is calculated by multiplying the number of “nodes” by the number of GPUs in each “node”. Understanding “world_size” is crucial for managing the number of processes and optimizing workload distribution in the system.
- **Process group**: In training or testing a model across multiple GPUs, a “process group” is a collection of processes corresponding to the number of GPUs being used. Each process in the “process group” represents a GPU, and these processes work together to perform parallel computations during model training.

`DistributedDataParallel` (DDP) is a method for module-level parallel processing in PyTorch and can run on multiple computers. It allows for model training across multiple GPUs and machines, accelerating the training process. Applications using DDP need to create multiple processes and instantiate a unique DDP version for each process. This ensures parallel processing and optimizes the performance of gradient synchronization and buffering between processes. The recommended approach for using DDP is to create one process for each model replica, where each model replica can span multiple computing devices.

To create a DDP module, we need to set up the process groups as follows:

```
import os
import sys
import tempfile
import torch
import torch.distributed as dist
import torch.nn as nn
import torch.optim as optim
import torch.multiprocessing as mp

from torch.nn.parallel import DistributedDataParallel as DDP

def setup(rank, world_size):
    os.environ['MASTER_ADDR'] = 'localhost'
    os.environ['MASTER_PORT'] = '12355'

    # initialize the process group
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def cleanup():
    dist.destroy_process_group()
```

Next, we will create a DataLoader using `torch.utils.data.distributed.DistributedSampler`. The `DistributedSampler` is a sampler in PyTorch used for distributing data when training across multiple GPUs or multiple machines. When using `DistributedSampler`, the entire dataset indices will be divided into `world_size` parts. After splitting the indices into parts, the `DistributedSampler` will distribute them evenly to the DataLoader in each process without any overlap. This ensures that each process receives a unique set of indices, independent of other processes.

```
from torch.utils.data.distributed import DistributedSampler
def prepare(rank, world_size, batch_size=32, pin_memory=False, num_workers=0):
    dataset = Your_Dataset()
    sampler = DistributedSampler(dataset, num_replicas=world_size, rank=rank, sh

    dataloader = DataLoader(dataset, batch_size=batch_size, pin_memory=pin_memor

    return dataloader
```

Assuming we have 3 GPUs and the length of the dataset is 10. The `DistributedSampler` will ensure that the indices are distributed evenly. In this case, the number of indices (10) does not divide evenly by the number of processes (3), but the `DistributedSampler` still ensures that the indices are distributed fairly and not skewed.

If we set `drop_last=False` when defining the `DistributedSampler`, it will automatically pad the extra elements. This happens when the number of indices is not divisible by the number of processes. For example, if `rank=1`, the `DistributedSampler` will split the indices `[0,1,2,3,4,5,6,7,8,9]` into `[0,3,6,9]` and automatically add a padding element (value 0) to ensure there are enough indices for each process. Conversely, if we set `drop_last=True`, it will drop the surplus elements. For example, if `drop_last=True` and `rank=1`, the `DistributedSampler` will split the indices `[0,1,2,3,4,5,6,7,8,9]` into `[0,3,6]` and drop the last element (9) to ensure the number of indices is divisible by the number of processes.

Next, we will wrap the model using `DistributedDataParallel`.

```
class RandomModel(nn.Module):
    def __init__(self):
        super(RandomModel, self).__init__()
        self.net1 = nn.Linear(10, 10)
        self.relu = nn.ReLU()
        self.net2 = nn.Linear(10, 5)

    def forward(self, x):
        return self.net2(self.relu(self.net1(x)))


def demo_basic(rank, world_size):
    print(f"Running basic DDP example on rank {rank}.")
    setup(rank, world_size)

    # create model and move it to GPU with id rank
    model = RandomModel().to(rank)
    ddp_model = DDP(model, device_ids=[rank])

    loss_fn = nn.MSELoss()
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.001)

    optimizer.zero_grad()
    outputs = ddp_model(torch.randn(20, 10))
    labels = torch.randn(20, 5).to(rank)
    loss_fn(outputs, labels).backward()
    optimizer.step()

    cleanup()
```

When you want to access custom attributes of a model that has been wrapped in `DistributedDataParallel` (DDP), you will need to refer to `model.module`. This means that the actual model is stored as a "module" attribute of the DDP model. Therefore, when you want to access attributes

`xxx` that are not built-in attributes or functions, you must access them through `model.module.xxx`.

Thus, if you want to load a saved DDP model into a non-DDP model, you need to manually remove the "module" prefix as follows:

```
model_dict = OrderedDict()
pattern = re.compile('module.')
for k,v in state_dict.items():
    if re.search("module", k):
        model_dict[re.sub(pattern, '', k)] = v
    else:
        model_dict = state_dict
model.load_state_dict(model_dict)
```

Next, we will call the `spawn` function to run the processes in parallel.

```
def run_demo(demo_fn, world_size):
    mp.spawn(demo_fn,
             args=(world_size,),
             nprocs=world_size,
             join=True)
```

In the `demo_fn` function used to start the distributed training process, the first parameter is `rank`. This parameter represents the current process's order in the `process group`. When using the `mp.spawn` function in PyTorch to start distributed training, `rank` will be automatically assigned to each process, and we do not need to pass it explicitly. By default, `rank=0` is considered the master node. This is usually the first process in the `process group`, responsible for coordinating and managing the other processes during the distributed training. The value of `rank` ranges from 0 to K-1, where K is the total number of processes (GPUs) in the `process group`. For example, if there are 2 GPUs (K=2), the ranks of the processes will be 0 and 1. `rank=0` will be the master node, and `rank=1` is the second process in the distributed training.

We can also simplify the DDP code by using PyTorch Elastic. Suppose we have an `elastic_ddp.py` file as follows:

```
import torch
import torch.distributed as dist
import torch.nn as nn
import torch.optim as optim

from torch.nn.parallel import DistributedDataParallel as DDP

class ToyModel(nn.Module):
    def __init__(self):
        super(ToyModel, self).__init__()
        self.net1 = nn.Linear(10, 10)
        self.relu = nn.ReLU()
        self.net2 = nn.Linear(10, 5)

    def forward(self, x):
        return self.net2(self.relu(self.net1(x)))


def demo_basic():
    dist.init_process_group("nccl")
    rank = dist.get_rank()
    print(f"Start running basic DDP example on rank {rank}.")

    # create model and move it to GPU with id rank
    device_id = rank % torch.cuda.device_count()
    model = ToyModel().to(device_id)
    ddp_model = DDP(model, device_ids=[device_id])

    loss_fn = nn.MSELoss()
    optimizer = optim.SGD(ddp_model.parameters(), lr=0.001)

    optimizer.zero_grad()
    outputs = ddp_model(torch.randn(20, 10))
    labels = torch.randn(20, 5).to(device_id)
    loss_fn(outputs, labels).backward()
    optimizer.step()

if __name__ == "__main__":
    demo_basic()
```

Using a single `torch run` command, we can run across all nodes to initialize a DDP job as follows:

```
torchrun --nnodes=2 --nproc_per_node=4 --rdzv_id=100 --rdzv_backend=c10d --rdzv_
```

With the command above, we run the DDP script on two hosts, with each host running 4 processes, meaning we are running on 8 GPUs. Note that `$MASTER_ADDR` must be the same across all nodes.

Conclusion

So we have gone through several methods to train a model on multiple GPUs. As models become larger, understanding how to train on GPUs becomes

increasingly important. 😊 The methods presented above are the fundamental ideas of distributed training; you can customize them for your model and data to achieve the best results.

Python | Pytorch | AI | Artificial Intelligence | Technology

Written by Lin Wang

1.5K Followers · 118 Following

Follow

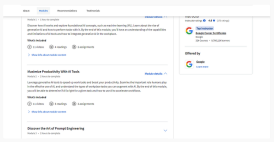
AI Engineer | Mobile Developer | Indie Hacker | My Youtube: https://www.youtube.com/@linwangg

No responses yet

What are your thoughts?

Respond

More from Lin Wang



Lin Wang

Is The Google AI Essentials Certificate ACTUALLY Worth It?

Google just came out with an AI related course, and when you complete it, you...

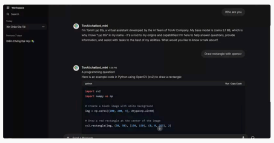
Jun 18, 2024 29

Lin Wang

Top 13 Micro SaaS Ideas—Profitable Examples for 2024

Note: If you don't have a Medium membership, you can read this story here.

Jul 8, 2024 153 5



Lin Wang

Create a free 'ChatGPT-like' Chatbot with Ollama and Open...

The power of ChatGPT has led you to explore large language models (LLMs) and want to...

Oct 3, 2024 4

Lin Wang

My Experience Building a Micro SaaS with ShipFast: A Review

Should you be interested in any of the mentioned products:

Jun 18, 2024 190 4

See all from Lin Wang

Recommended from Medium

Aditya Shanmugham

Using DeepSpeed and FSDP with Accelerate—Part 1

A Comprehensive Guide to DeepSpeed and Fully Sharded Data Parallel (FSDP) with...

Nov 19, 2024 13 1

Minyang Chen

Multi-GPU Training for Llama 3.2 using DeepSpeed and Redundanc...

For inference tasks, it's preferable to load entire model onto one GPU, containing all...

Oct 1, 2024 63 1

Lists

AI Regulation
6 stories · 693 saves

ChatGPT prompts
51 stories · 2586 saves

Generative AI Recommended Reading
52 stories · 1661 saves

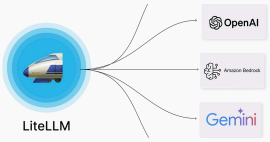
ChatGPT
21 stories · 975 saves

Mohamed Amri

How to serve Deepseek flagship models for inference with vLLM a...

One single line of code. That's all it takes to serve Deepseek models (or any open-sourc...

Feb 3 · 28 · 2

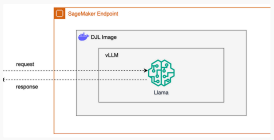


Henry Navarro

LiteLLM : the orchestrator you need for your LLM application ...

Empowering Seamless Integration and Security in the Age of LLMs

Oct 23, 2024 · 30

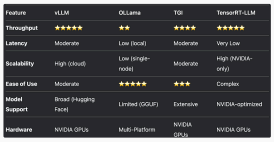


In TDS Archive by Jake Teo

Deploying your Llama Model via vLLM using SageMaker Endpoint

Leveraging AWS's MLOps platform to serve your LLM models

Sep 12, 2024 · 88



Feature	vLLM	OLLama	TGI	TensorRT-LLM
Throughput	★★★★★	★★	★★★★	★★★★★
Latency	Moderate	Low (local)	Moderate	Very Low
Scalability	High (cloud)	Low (single-node)	Moderate	High (NVIDIA-only)
Ease of Use	Moderate	★★★★★	★★★	Complex
Model Support	Broad (Hugging Face)	Limited (GGUF)	Extensive	NVIDIA-optimized
Hardware	NVIDIA GPUs	Multi-Platform	NVIDIA GPUs	NVIDIA GPUs

Abhishek Gupta

vLLM vs. OLLama and Competitors: A Comprehensive Guide to LLM...

Introduction The surge in large language model (LLM) adoption has intensified the...

Feb 1 · 1

See more recommendations